

CoMEMBENCH: BENCHMARKING COLLABORATIVE MEMORY BOUNDARIES ACROSS MULTI-AGENT WORKFLOW TOPOLOGIES

Sen Zhao^{1*} Ruiqi Kong^{1*} Zuyu Zhang¹ Lifeng Shen²
Xinyu He⁴ Ding Zou⁵ Xu Zhang^{1†} Qinghua Zhang³

¹Academy of Advanced Interdisciplinary Studies,
Chongqing University of Posts and Telecommunications, Chongqing, China

²School of Artificial Intelligence,
Chongqing University of Posts and Telecommunications, Chongqing, China

³School of Computer Science and Technology,
Chongqing University of Posts and Telecommunications, Chongqing, China

⁴Towngas, China

⁵Intelligent System Department, Zhongxing Telecom Equipment (ZTE),
Changsha, Hunan, China

ABSTRACT

Multi-agent workflows require task-relevant information to be shared across agents, while irrelevant, stale, unverified, or incompatible information must remain isolated. We call this task-conditioned scope of information a *collaborative memory boundary*. Workflow topology determines which intermediate artifacts are applicable to which downstream workers and when they cease to be valid, thereby providing a structural stress dimension for sharing and isolation. Existing memory benchmarks primarily evaluate retention and retrieval, whereas multi-agent benchmarks emphasize coordination and end-to-end completion, leaving topology-conditioned memory boundaries largely unmeasured. We introduce CoMEMBENCH, an execution-grounded benchmark for collaborative memory sharing and isolation across multi-agent workflow topologies. It constructs 800 composite workflows across four domains from source-grounded dependency graphs, with node-local specifications, verifiable artifact handoffs, native evaluators, and matched isolation challenges. CoMEMBENCH measures workflow completion, verified node progress, required-handoff reliability, isolation robustness, and token cost. Experiments reveal a sharing-isolation trade-off: broader context improves information availability but can weaken isolation, while system rankings shift across topologies and artifact violations. CoMEMBENCH is released at <http://comembench.world>.

1 INTRODUCTION

Large language model agents increasingly tackle complex tasks through teams of specialized workers (Zhu et al., 2025; Ke et al., 2026). Because workers operate with different instructions, observations, and execution histories, they maintain distinct local task memories, regardless of implementation. Interdependent tasks therefore require applicable information to be shared with the agents that need it, while irrelevant, mismatched, stale, or unverified information remains isolated. Insufficient sharing withholds required evidence; failed isolation allows memory pollution to alter otherwise valid executions. We call this task-conditioned separation a *collaborative memory boundary*.

The required boundary is determined by task dependencies, rather than by agent identity alone. Figure 1 grounds this idea in a repository maintenance workflow. A chain passes one required result to its successor; a split-join keeps branch-local states separate before merging their outputs; reuse exposes one verified artifact to multiple valid consumers while retaining producer-local traces; overlap

*Equal contribution.

†Corresponding author: zhangx@cqupt.edu.cn.

shares only the artifact common to partially intersecting contracts; and bounded revision replaces a superseded result before verification. The same artifact can therefore be required by one consumer, inapplicable to another, or valid only at a particular workflow state. Our central question is whether a multi-agent system can maintain these changing sharing and isolation boundaries throughout execution.

Existing benchmarks cover complementary pieces of this question but not their intersection. They separately study long-context memory, governed or privacy-constrained group memory, multi-agent orchestration, and executable tool use (Maharana et al., 2024; Wu et al., 2025; Hu et al., 2026a; Yang et al., 2026a; Ren et al., 2026; Park et al., 2026; Zhu et al., 2025; Shen et al., 2024; Ke et al., 2026; Li et al., 2026a). As Table 1 shows, none jointly provides explicit multi-agent workflows with diverse task-dependency topologies, verifiable artifact handoffs, and separate sharing and isolation evaluation under executable conditions. A checkmark denotes a capability explicitly instantiated and evaluated. End-to-end success alone therefore cannot reveal whether required memory was unavailable or inapplicable memory crossed a task boundary.

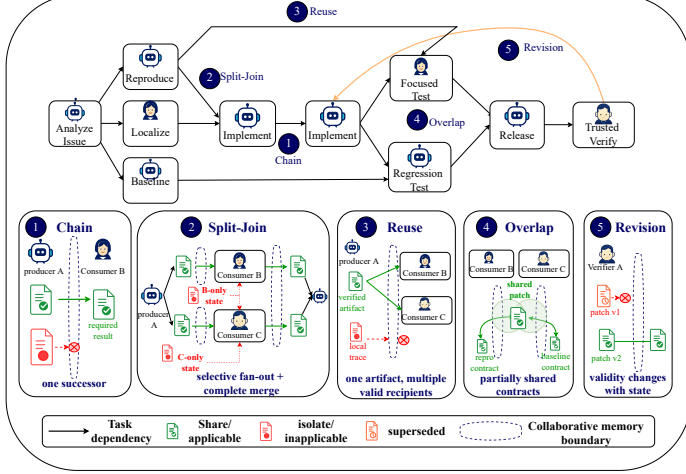


Figure 1: **Topology-conditioned memory boundaries.** A repository workflow instantiates five dependency motifs, each changing which artifacts downstream agents require, must not receive, or must treat as superseded.

We introduce COMEMBENCH, an execution-grounded benchmark of 800 clean workflows across four domains. It recovers source-grounded dependency graphs and extracts coherent executable subgraphs while preserving task requirements, entities, resources, and evaluators; nodes have local specifications and declared inputs, and edges carry required artifacts. Each workflow is paired with a matched challenge that introduces one inapplicable artifact while holding the task, graph, and non-target inputs fixed. We evaluate terminal success, verified node progress, handoff reliability, and isolation robustness across domains, topologies, and contamination types, without constraining system organization or internal memory. Experiments show that progress, handoff reliability, and isolation diverge: broader context may improve progress while weakening isolation, and system rankings shift across structural conditions. Collaborative memory boundaries are therefore a distinct systems property, not a proxy for task success or retrieval quality.

2 RELATED WORK

2.1 AGENT MEMORY AND INFORMATION BOUNDARIES

Memory evaluation has progressed from static evidence use in long contexts and fixed corpora (Bai et al., 2024; Hsieh et al., 2024; Petroni et al., 2021; Thakur et al., 2021) to persistent and interactive memory. MemoryBank, LoCoMo, LongMemEval, RealTalk, and StoryBench evaluate long-term or multi-session recall and reasoning (Zhong et al., 2024; Maharana et al., 2024; Wu et al., 2025; Lee et al., 2025; Wan & Ma, 2025); MemoryAgentBench further considers incremental ingestion, test-time learning, and selective forgetting (Hu et al., 2026a). These benchmarks move beyond static reading, but predominantly evaluate memory accumulated and used by one focal agent rather than its selective availability across agents executing interdependent tasks.

Table 1: Comparison with related benchmarks.

Benchmark	Multi-Agent Workflow	Workflow Topology Diversity	Artifact Handoffs	Sharing Evaluation	Isolation Evaluation	Containerized Execution
LoCoMo (Maharana et al., 2024)	—	—	—	—	—	—
LongMemEval (Wu et al., 2025)	—	—	—	—	—	—
MemoryAgentBench (Hu et al., 2026a)	—	—	—	—	—	—
GroupMemBench (Yang et al., 2026a)	—	—	—	—	—	—
GateMem (Ren et al., 2026)	—	—	—	—	✓	—
PAC-Bench (Park et al., 2026)	✓	—	—	—	✓	—
PeopleJoin (Jhamtani et al., 2025)	—	—	—	—	—	—
SILO-BENCH (Zhang et al., 2026b)	✓	✓	✓	—	—	—
MultiAgentBench (Zhu et al., 2025)	✓	—	—	—	—	—
TaskBench (Shen et al., 2024)	—	✓	—	—	—	—
MASBench (Ke et al., 2026)	✓	✓	✓	—	—	—
Toolathlon (Li et al., 2026a)	—	—	—	—	—	✓
COMEMBENCH	✓	✓	✓	✓	✓	✓

Recent benchmarks extend this question to multi-party and governed settings. GroupMemBench evaluates speaker-grounded memory (Yang et al., 2026a); GateMem evaluates utility, access control, and active forgetting in a shared memory pool (Ren et al., 2026); and PAC-Bench studies private memories under two-agent privacy constraints (Park et al., 2026). They establish that appropriate disclosure matters in addition to recall, but define memory scope primarily through speakers, identities, roles, or access policies. COMEMBENCH instead asks which artifact a consumer requires now and which plausible artifact must remain outside its context. Together, sharing and isolation define a collaborative memory boundary shaped by task dependencies, verification, and version.

2.2 MULTI-AGENT AND WORKFLOW BENCHMARKS

Multi-agent benchmarks increasingly study coordination structure and task dependencies. Agent-Bench, MindAgent, and LLM-Coordination evaluate agents in interactive or controlled collaborative settings (Liu et al., 2024; Gong et al., 2024; Agashe et al., 2025); MultiAgentBench varies star, chain, tree, and graph *interaction protocols* (Zhu et al., 2025). TaskBench and MASBench instead vary task dependencies, including chains, DAGs, dependency depth, aggregation breadth, and parallelism (Shen et al., 2024; Ke et al., 2026). These works establish the importance of organization and workflow structure, but neither interaction topology nor end-to-end task success reveals whether local memory was selectively exposed according to task dependencies.

Execution benchmarks make this distinction testable. τ -bench and AppWorld provide stateful executable outcomes (Yao et al., 2025; Trivedi et al., 2024), while Toolathlon and Toolathlon-GYM provide long-horizon application workflows and reproducible environments (Li et al., 2026a; Zhang et al., 2026a). PeopleJoin and SILO-BENCH further study collaboration under distributed information (Jhamtani et al., 2025; Zhang et al., 2026b). Existing workflow benchmarks vary coordination or task structure but primarily score end-to-end outcomes, while executable agent benchmarks lack controlled boundary tests. COMEMBENCH extracts topology-diverse workflows from source task graphs, validates them natively, and tests typed, versioned handoffs with matched clean-polluted runs. Behavioral scoring separates completion, progress, sharing, and isolation.

3 THE COMEMBENCH BENCHMARK

COMEMBENCH represents each instance as an executable task graph with node-local specifications, typed and version-aware handoffs, resources, and a domain-native evaluator. Its pipeline recovers domain task graphs, extracts topology-controlled workflows, adds matched isolation challenges, and certifies their execution. Figure 2 summarizes the design.

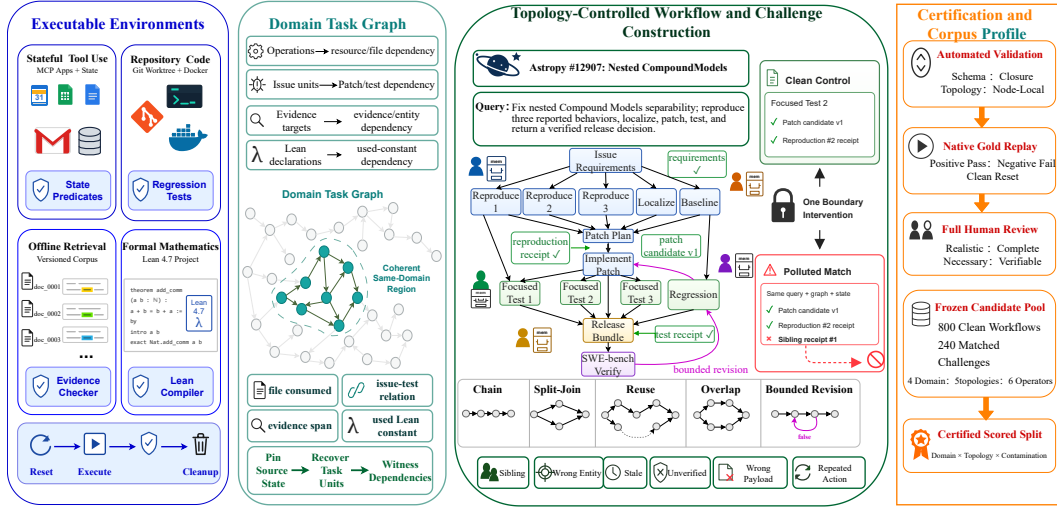


Figure 2: Overview of the COMEMBENCH construction pipeline.

3.1 WORKFLOW AND BOUNDARY FORMULATION

An executable workflow comprises a request q , task graph $G = (V, E)$, node-local specifications $\mathcal{X} = \{x_v\}_{v \in V}$, resources and initial state $\mathcal{R}$, and a domain-native evaluator Φ :

$$\mathcal{W} = (q, G, \mathcal{X}, \mathcal{R}, \Phi). \quad (1)$$

Nodes denote required work and dependencies, not physical agents or memory modules. Each edge $e = (u, v)$ defines a handoff $h_e = (u, v, \tau, z, \nu, \sigma)$, recording semantic type, entity scope, version, and verification state; artifacts include test or proof receipts, evidence spans, and resource identifiers.

The contract partitions contexts into those where h is applicable, $C^+(h)$, or inapplicable, $C^-(h)$ —the *collaborative memory boundary*. Sharing delivers a current, verified artifact to $C^+(h)$; isolation prevents an incompatible artifact from affecting $C^-(h)$. Because internal memory is not comparable across architectures, COMEMBENCH scores artifact exposure, verified downstream execution, and matched clean–polluted runs.

3.2 EXECUTABLE ENVIRONMENTS

COMEMBENCH executes 800 workflows in domain-native environments. Across 200 workflows per domain, stateful tasks draw from 25 pinned MCP servers detailed in Appendix D.1, activating only the 4–10 declared per task; code tasks span 12 repositories and use issue-specific Docker environments evaluated by the official SWE-bench harness against fixed base commits and FAIL-TO-PASS/PASS-TO-PASS tests; retrieval tasks use 200 immutable corpus snapshots containing 16,335 document records, with document-level and exact-span verification; and mathematical tasks compile submitted proofs in isolated copies of three commit-locked Lean 4.7 projects. These native evaluators produce node-level receipts and terminal workflow verdicts.

Each run follows a prepare–execute–verify–cleanup protocol bound to pinned assets, environment configurations, source revisions, and evaluator versions. Stateful tasks additionally record state digests and cleanup receipts, while infrastructure outcomes remain separate from method failures.

3.3 RECOVERING DOMAIN TASK GRAPHS

From sources with realistic requirements and executable evaluators, we recover task units and dependencies in four domains: stateful tool use, repository code change, offline retrieval, and formal mathematics. Source instances provide requirements, assets, and evaluators rather than ready-made multi-agent workflows. We pin assets by task identifier, commit, corpus snapshot, or environment digest; shared entities, resources, state transitions, and evaluator contracts then define graph edges.

Recovery follows each source’s native structure. We align Toolathon operations with resources, initialization, and evaluator predicates (Li et al., 2026a; Zhang et al., 2026a); expand SWE-bench

Verified issues into reproduction, localization, patch, and test units (Jimenez et al., 2024); decompose BrowseComp+ items into evidence targets, entity constraints, and synthesis (Chen et al., 2025); and recover declaration graphs and used constants from three miniCTX Lean 4.7 projects (Hu et al., 2024). We exclude declarations transitively depending on `sorryAx` and retain only edges witnessed by consumed files, issue–test links, evidence requirements, or Lean constants.

3.4 TOPOLOGY-CONTROLLED WORKFLOW AND CHALLENGE CONSTRUCTION

Topology-controlled workflow extraction. Within each domain graph, we select a connected, scenario-coherent subgraph with one terminal objective, complete dependency closure, necessary witnessed edges, and native-evaluator coverage. We assign topology labels from explicit graph witnesses: *chain* denotes a serial dependency; *split-join*, independent branches that reconverge; *reuse*, one typed artifact consumed by multiple downstream regions; *overlap*, consumers with partially shared predecessor sets; and *bounded revision*, one verifier-triggered repair and superseding artifact. Labels are multi-label because a workflow may contain several motifs.

Query and node-local realization. For each extracted graph G_i , we construct a natural request q_i and a local task x_v for every node. The request specifies the objective, entities, resources, deliverables, constraints, and acceptance conditions without prescribing an execution plan, following Toolathlon’s distinction between realistic underspecification and information required for deterministic execution (Li et al., 2026a). Each node satisfies *node-local closure*: its task, authorized environment view, and declared incoming artifacts contain the information required for execution. Outgoing artifacts specify semantic type, entity scope, version, and verification requirement, making the sharing boundary executable rather than implicit.

Matched isolation challenges. For every clean workflow, we select a consumer boundary with a local topology witness and expose exactly one task-inapplicable artifact to its target consumer; the injection plan remains private. Six operators instantiate concrete violations: *sibling crosswire* routes an artifact from the wrong branch; *incompatible payload* violates its semantic contract; *unverified result* lacks a required receipt; *completed-as-pending* repeats a completed side effect; *wrong entity* binds a valid type to the wrong resource; and *stale version* exposes a superseded artifact. Topology identifies *where* the boundary occurs, while the operator specifies *how* it is violated. Split-joins support crosswires, bounded revisions support stale versions, and chain, reuse, and overlap expose handoff, entity, or verification mismatches; Figure 3 reports the resulting supported combinations. Each pair otherwise shares its query, graph, environment binding, tools, and non-target inputs. We score the polluted target only when the corresponding clean node has a verified receipt, so Isolation Challenge Success measures robustness at an otherwise solvable boundary.

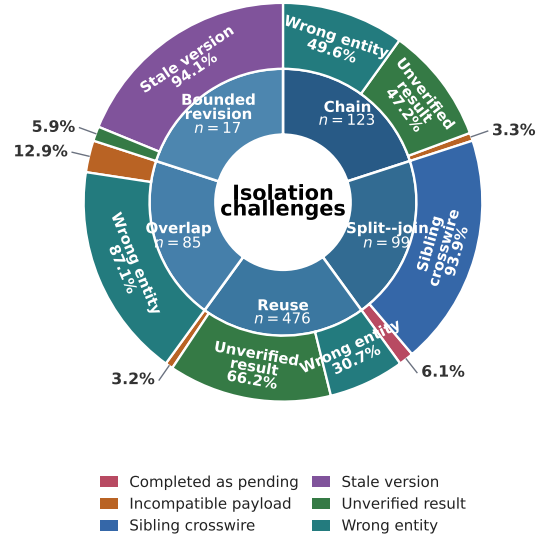


Figure 3: Topology-operator challenge distribution.

3.5 CERTIFICATION AND CORPUS PROFILE

Certification. A workflow enters the scored split only after three gates. Automated checks enforce source locks, graph closure, acyclicity outside declared revision edges, topology witnesses, node-local closure, control-polluted equality, evaluator coverage, and removal of private material. A private *Gold execution plan* then runs the workflow from a clean reset through the released harness and evaluator; targeted negative mutations must fail, while repeated executions must produce equivalent receipts and semantic state digests. Finally, human reviewers assess query plausibility and completeness,

Table 2: Structural demand and multi-label topology coverage.

Domain	Structural demand (mean)							Execution-topology coverage (count)					
	N	Tasks	Depth	Width	Handoffs	Fan-outs	Joins	Chain	Split-join	Reuse	Overlap	Revision	Types/WF
Stateful tool use	200	7.78	6.02	2.30	17.17	4.62	4.75	200	200	199	197	70	4.33
Repository code	200	11.58	6.00	4.78	19.97	3.81	4.81	200	200	200	200	74	4.37
Offline retrieval	200	9.35	8.35	2.00	19.04	6.35	3.94	200	200	200	200	126	4.63
Formal mathematics	200	11.70	4.59	5.58	14.47	2.88	3.56	200	200	200	190	144	4.67
All / Avg.	800	10.10	6.24	3.66	17.66	4.41	4.26	800	800	799	787	414	4.50

node-local executability, edge necessity, and evaluator–goal alignment. Rejected workflows are repaired and recertified or excluded, and infrastructure defects quarantine the affected family rather than count as method failures. Audit records retain each decision and its receipts, source-grouped splits limit near-duplicate leakage, and Gold plans never enter prompts, public traces, or scores.

Corpus profile. The certified release contains 800 clean workflows, evenly distributed across four domains, and 800 matched isolation challenges. Table 2 summarizes per-workflow structural demand and multi-label topology coverage: a workflow contributes every motif present in its graph, whereas receipt-routed conditional revisions are counted as revision boundaries rather than ordinary source-graph edges. Across the release, a workflow averages 10.10 executable task units, 17.66 required handoffs, and 4.50 topology types. The four domains pair these structures with distinct executable artifacts—mutable application state, patches and test evidence, corpus evidence spans, and proof terms with compiler receipts—so the profile captures both graph-level coordination demand and domain-native verification rather than graph size alone.

4 EVALUATION PROTOCOL

We evaluate observable consequences of collaborative memory management rather than hidden internal states, using task graphs, declared handoffs, exposure logs, verifier receipts, and native evaluators. **SR** measures clean-workflow terminal success. **Verified Node Completion Rate (VNCR)**, $|C|^{-1} \sum_{q \in C} |\mathcal{V}_q|/|\mathcal{U}_q|$, is the macro-average verified fraction of required nodes; **Verified Handoff Success (VHS)**, $|\mathcal{H}^+|/|\mathcal{H}|$, measures success over eligible consumers; and **Isolation Challenge Success (ICS)**, $|\mathcal{P}^+|/|\mathcal{P}|$, measures polluted-target success over matched pairs whose corresponding clean target succeeds. Together, these measures distinguish terminal completion, verified progress, required sharing, and isolation.

Conditional metrics are computed only over eligible cases. Results are reported by domain and, in dedicated analyses, by target topology and pollution operator. Benchmark and infrastructure failures are separated from method failures, cost is analyzed in Section 5.5, and formal eligibility and accounting rules appear in Appendix F.

5 EXPERIMENTS

We compare multi-agent systems across domains and backbones, isolate memory mechanisms under a fixed executor, analyze results by topology and pollution operator, and measure their runtime costs.

5.1 EXPERIMENTAL SETUP

Evaluated systems. We organize ten baselines by their evaluated collaboration abstraction, from general-purpose agent execution to increasingly specialized multi-agent organization. *General-purpose agent runtimes*: Codex and Hermes (OpenAI, 2026; Nous Research, 2026); *multi-agent frameworks*: Deep Agents, CrewAI, and AutoGen (LangChain AI, 2026; CrewAI, Inc., 2026; Wu et al., 2024); *fixed collaboration methods*: Peer Review, SMoA, and OMAC (Xu et al., 2023; Li et al., 2025; 2026b); and *automated MAS design*: ADAS and EvoMAS (Hu et al., 2025; 2026b). Each receives the same node-local task, authorized tools, and declared inputs while retaining its own routing, scheduling, and memory decisions; Appendix E documents the evaluated configurations.

Table 3: System and backbone results.

Backbone	System	Stateful Tool			Repository Code			Offline Retrieval			Formal Mathematics			ICS
		SR	VNCR	VHS	SR	VNCR	VHS	SR	VNCR	VHS	SR	VNCR	VHS	
Qwen3.8-27B	Codex	0.0	5.1	32.7	0.0	33.2	43.0	4.1	23.3	10.7	0.0	9.1	5.3	0.0
	Hermes	0.0	3.2	18.2	0.0	25.2	30.2	4.1	35.7	14.4	0.0	6.2	2.0	0.0
	Deep Agents	0.0	5.5	1.4	0.0	57.7	66.8	2.0	37.8	9.5	0.0	11.4	3.4	38.1
	CrewAI	0.0	38.3	67.1	0.0	44.1	54.8	12.0	53.1	38.1	0.0	10.8	9.3	12.9
	AutoGen	0.0	33.9	56.8	0.0	48.8	55.1	18.0	50.7	37.3	0.0	10.3	8.7	20.0
	Peer Review	0.0	34.9	61.8	4.0	49.7	55.1	0.0	24.3	5.0	0.0	14.0	11.5	19.1
	SMoA	0.0	43.8	72.4	0.0	44.5	51.0	16.0	53.3	45.3	0.0	13.3	4.7	10.1
	OMAC	0.0	31.8	55.3	0.0	52.9	62.1	8.0	39.9	38.9	0.0	7.4	5.8	17.3
	ADAS	0.0	18.3	36.6	0.0	17.0	37.3	0.0	12.5	0.0	0.0	6.5	8.2	7.4
	EvoMAS	0.0	21.0	39.0	4.0	67.6	73.2	0.0	13.6	0.0	0.0	19.4	10.8	26.8
DeepSeek-V4-Flash	Codex	0.0	2.7	0.0	0.0	17.1	33.2	0.0	5.1	0.0	0.0	9.5	3.9	0.0
	Hermes	0.0	5.6	9.4	0.0	26.5	26.1	2.0	45.7	17.1	0.0	4.2	0.0	0.0
	Deep Agents	0.0	4.3	1.8	0.0	18.8	26.1	0.0	5.3	0.0	0.0	1.4	0.0	0.0
	CrewAI	0.0	14.4	29.6	0.0	40.1	45.0	2.0	33.7	9.7	0.0	9.2	4.1	6.1
	AutoGen	0.0	15.0	28.8	0.0	63.8	71.9	4.0	40.5	24.8	0.0	10.2	6.9	15.6
	Peer Review	0.0	17.6	24.0	0.0	52.0	65.2	2.0	23.3	17.2	0.0	9.9	6.0	17.1
	SMoA	0.0	21.5	51.9	2.0	76.8	85.8	2.0	41.0	32.9	0.0	8.2	3.2	14.9
	OMAC	0.0	66.2	82.5	4.0	84.2	89.4	10.0	65.5	57.6	0.0	12.3	5.3	4.4
	ADAS	0.0	17.8	52.0	8.0	67.6	80.9	4.0	29.9	16.1	0.0	11.5	2.0	14.3
	EvoMAS	0.0	12.9	22.2	6.0	58.8	68.9	0.0	13.6	4.1	0.0	17.6	9.6	9.8
GPT-6-Luna	Codex	0.0	6.0	2.0	0.0	13.3	17.2	0.0	3.5	0.0	0.0	4.0	0.0	0.0
	Hermes	0.0	6.0	43.8	0.0	25.9	25.4	0.0	31.4	11.5	0.0	14.7	0.0	0.0
	Deep Agents	0.0	3.0	0.0	0.0	12.5	32.1	0.0	8.9	0.0	0.0	0.0	0.0	0.0
	CrewAI	0.0	35.7	63.1	0.0	52.5	64.4	0.0	41.6	8.2	0.0	8.4	6.0	13.3
	AutoGen	0.0	19.0	69.2	0.0	37.6	73.3	4.0	41.6	31.1	0.0	12.0	2.4	11.8
	Peer Review	0.0	13.7	35.8	0.0	3.1	20.0	0.0	10.2	0.0	0.0	4.0	0.0	0.0
	SMoA	0.0	29.6	68.5	0.0	49.2	64.1	2.0	43.5	17.0	0.0	11.2	2.5	15.1
	OMAC	0.0	39.7	66.9	0.0	35.3	72.4	0.0	28.5	12.5	0.0	7.8	1.4	4.4
	ADAS	0.0	26.8	69.8	0.0	26.7	31.3	2.0	36.0	9.4	0.0	6.5	7.4	7.1
	EvoMAS	0.0	43.6	63.7	0.0	55.5	67.9	0.0	17.0	5.2	0.0	18.0	16.4	7.0

Controlled memory comparison. To isolate memory, we fix a topology-preserving supervisor-worker executor and vary only how information crosses workers. We compare LOCAL-ONLY, FULL CONTEXT, SHARED FLAT RAG, Mem0 (Chhikara et al., 2025), A-Mem (Xu et al., 2025), and PlugMem (Yang et al., 2026b); prompts, workers, tools, graph, evaluator, budgets, and retries remain fixed. Appendix E specifies each implementation.

Backbones and protocol. We use Qwen3.8-27B, DeepSeek-V4-Flash, and GPT-6-Luna on a fixed domain-stratified split, while the controlled memory comparison fixes Qwen3.8-27B. Selected configurations are repeated to measure run-to-run variability. Clean and polluted runs use isolated task-scoped environments and the same declared consumer boundary. Each run manifest records the backbone, endpoint, context limit, decoding policy, and framework version; unsupported system-backbone combinations are marked “-”. Scoring, eligibility, failure attribution, and token accounting follow Section 4 and Appendix F.

5.2 SYSTEM AND BACKBONE COMPARISON

Table 3 is the primary system-level comparison. It reports SR, VNCR, and VHS separately for each domain, together with ICS.

All entries are percentages. Metric definitions and scoring details are given in Section 4 and Appendix F.

Task success and verified progress. Terminal completion remains difficult across all three backbones: no supported configuration solves a Stateful Tool or Formal Mathematics workflow, and the best SR is only 8.0 in Repository Code and 18.0 in Offline Retrieval. This does not imply an absence of useful collaboration. Some zero-SR configurations still reach 84.2 VNCR and 89.4 VHS, showing that they verify substantial parts of a workflow and execute consumers after valid predecessors, yet

Table 4: Memory-mechanism results under controlled multi-agent execution.

Memory mechanism	Stateful Tool				Repository Code				Offline Retrieval				Formal Mathematics			
	SR	VNCR	VHS	ICS	SR	VNCR	VHS	ICS	SR	VNCR	VHS	ICS	SR	VNCR	VHS	ICS
<i>Controls</i>																
LOCAL-ONLY	0.0	74.6	84.0	97.1	0.0	77.1	89.5	89.4	2.0	63.6	83.9	60.9	0.0	13.6	26.9	0.0
FULL CONTEXT	2.0	86.9	86.7	72.4	16.0	61.7	89.1	71.9	6.0	77.1	88.6	80.6	0.0	11.8	26.8	0.0
SHARED FLAT RAG 2.0	2.0	86.9	86.4	70.6	18.0	72.4	90.8	73.9	8.0	74.1	88.4	75.0	0.0	14.4	29.7	20.0
<i>External memory systems</i>																
MEM0	0.0	83.8	85.5	86.3	2.0	77.9	89.6	89.1	4.0	64.9	84.7	73.9	0.0	12.2	22.7	52.4
A-MEM	4.0	85.8	86.6	87.9	18.4	58.2	88.4	63.9	6.0	71.8	85.6	71.9	0.0	12.3	26.2	0.0
PLUGMEM	0.0	83.1	85.1	90.0	0.0	68.2	88.9	91.3	4.0	56.9	83.9	52.9	0.0	14.2	29.0	33.3

fail to close the terminal objective. The resulting gap establishes that workflow completion, verified progress, and required-handoff success capture distinct failure stages rather than interchangeable notions of task success.

Sharing versus isolation. Organization, backbone capability, required sharing, and isolation are likewise non-substitutable. General-purpose runtimes are generally among the weakest in verified progress, while more explicit multi-agent organizations improve particular domains but do not dominate across metrics. With Qwen3.8-27B, SMoA reaches 43.8/72.4 VNCR/VHS on Stateful Tool but only 10.1 ICS; Deep Agents shows the opposite profile, attaining 38.1 ICS despite weak progress. With DeepSeek-V4-Flash, OMAC leads progress in Stateful Tool, Code, and Retrieval but reaches only 4.4 ICS, below Peer Review’s 17.1. Rankings shift again under GPT-6-Luna. Because ICS conditions on a successful clean target, these reversals show that stronger execution and coordination do not automatically maintain the boundary around an otherwise solvable consumer.

5.3 MEMORY MECHANISM COMPARISON

Table 4 isolates this trade-off by fixing the executor and changing only the memory policy. LOCAL-ONLY still receives benchmark-declared handoffs; it disables additional cross-worker memory rather than required task edges. Relative to this control, FULL CONTEXT raises Stateful Tool VNCR from 74.6 to 86.9 and Code SR from 0.0 to 16.0, but reduces ICS from 97.1 to 72.4 and from 89.4 to 71.9 in the same domains. Broader exposure therefore improves availability in some workflows while making inapplicable artifacts harder to contain. SHARED FLAT RAG narrows exposure semantically and reaches 90.8 Code VHS, yet its Code ICS remains 73.9: relevance alone does not establish recipient, entity, verification, or version validity.

Structured stores also fail to dominate the three outcome levels. A-Mem reaches 18.4 Code SR but only 63.9 ICS, whereas PlugMem preserves 91.3 Code ICS with 0.0 Code SR and falls to 52.9 ICS on Retrieval. Thus retention, linking, and retrieval can improve particular domains without consistently enforcing the task-conditioned exposure decision measured here. Formal Mathematics ICS remains inconclusive because eligible clean targets are sparse. Overall, the controlled comparison supports the benchmark’s central distinction: making information available and keeping it applicable are separate memory-management requirements.

5.4 TOPOLOGY AND ISOLATION ANALYSIS

Tables 5 and 6 are offline stratified summaries of the traces reported in Table 3; they introduce no additional evaluation runs. Table 5 gives Qwen3.8-27B system-level results by the local topology witness, whereas Table 6 groups Qwen3.8-27B ICS by how the injected artifact violates applicability. The former identifies where a boundary is stressed; the latter identifies the failure mode applied there. VNCR is pooled over canonical nodes, VHS over eligible consumers, and ICS over matched pairs whose clean target succeeds. A workflow is therefore not assigned wholesale to every motif appearing in its graph. All entries are percentages.

Table 5: Topology-stratified task progress and isolation success.

System	Chain			Split-join			Reuse			Overlap			Bounded revision		
	VNCR	VHS	ICS	VNCR	VHS	ICS	VNCR	VHS	ICS	VNCR	VHS	ICS	VNCR	VHS	ICS
Codex	29.7	62.3	0.0	20.2	29.1	0.0	22.8	25.9	0.0	21.9	29.6	0.0	20.7	31.2	0.0
Hermes	28.4	56.5	0.0	20.1	20.5	–	22.7	14.8	0.0	21.4	19.8	–	25.0	0.0	–
Deep Agents	42.4	81.3	33.3	33.4	40.1	42.9	38.3	39.2	28.6	36.1	40.4	80.0	32.5	30.8	–
CrewAI	45.7	81.1	20.0	40.0	49.7	0.0	47.3	49.6	16.2	42.8	50.0	0.0	43.3	47.5	0.0
AutoGen	47.9	80.9	33.3	39.9	46.8	25.0	46.9	46.6	19.4	42.6	47.0	0.0	43.3	52.2	0.0
Peer Review	46.6	85.1	44.4	32.5	43.1	0.0	36.5	43.0	14.3	34.9	43.9	0.0	28.1	53.4	20.0
SMoA	47.0	79.5	25.0	41.8	51.7	0.0	49.4	53.8	11.9	44.1	52.2	0.0	44.6	55.0	0.0
OMAC	45.3	84.4	28.6	36.5	51.0	16.7	43.3	50.3	13.3	39.1	51.1	25.0	33.9	38.1	20.0
ADAS	16.5	56.7	0.0	14.2	26.9	33.3	16.4	25.3	0.0	15.4	27.7	0.0	14.5	42.6	0.0
EvoMAS	46.6	79.3	30.0	33.8	51.8	72.7	37.3	54.8	14.3	36.1	52.8	0.0	20.3	26.7	0.0

Table 6: Isolation by pollution operator.

System	Sibling crosswire	Incompatible payload	Unverified result	Completed as pending	Wrong entity	Stale version
Codex	0.0	0.0	0.0	–	0.0	0.0
Hermes	–	–	0.0	–	–	–
Deep Agents	42.9	0.0	31.0	–	80.0	–
CrewAI	0.0	0.0	22.2	0.0	0.0	0.0
AutoGen	33.3	0.0	26.3	0.0	0.0	0.0
Peer Review	0.0	0.0	29.6	0.0	0.0	20.0
SMoA	0.0	0.0	17.5	0.0	0.0	0.0
OMAC	33.3	0.0	20.7	0.0	12.5	20.0
ADAS	33.3	0.0	0.0	–	0.0	0.0
EvoMAS	80.0	16.7	20.7	0.0	0.0	0.0

Topology first identifies where the boundary must be maintained. Chain consumers have the highest VHS for every evaluated system, whereas branch separation, multi-consumer reuse, partial overlap, and revision introduce progressively different routing and validity requirements. The effect is not a uniform difficulty ordering: EvoMAS reaches 72.7 ICS on split-join but 14.3 on reuse, while Deep Agents reaches 80.0 on overlap but 28.6 on reuse. These reversals indicate topology-specific strengths rather than a single global isolation capability. They remain descriptive rather than causal comparisons.

Pollution operators provide the complementary semantic diagnosis: topology locates the boundary, while the operator specifies why the offered artifact is inapplicable there. EvoMAS handles 80.0 of eligible sibling-crosswire challenges but reaches only 20.7 on unverified results and zero on three other observed operators. Deep Agents reaches 80.0 on wrong-entity challenges but only 31.0 on unverified results. More broadly, incompatible payload and completed-as-pending challenges are almost never resisted, and unverified-result robustness remains low across systems. The two stratifications therefore show that collaborative-memory failure depends both on the dependency boundary under stress and on the artifact property that violates it; they are complementary diagnostics rather than evidence of a causal topology-operator interaction.

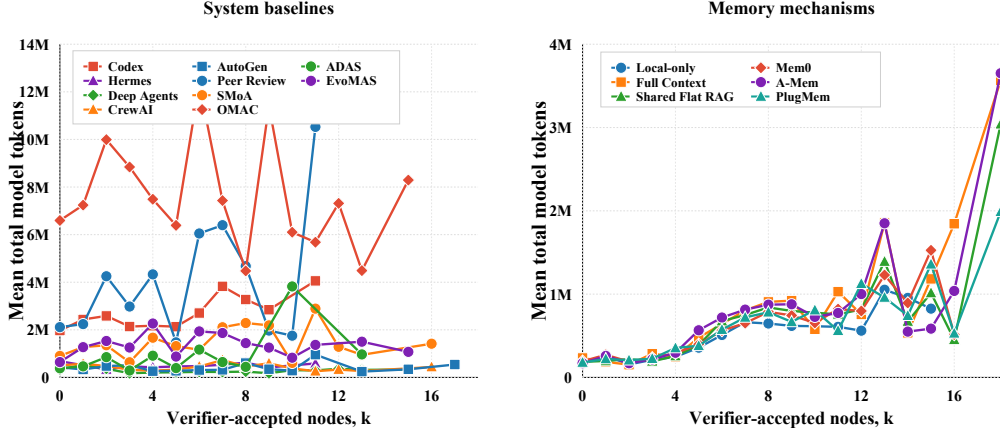


Figure 4: Progress-conditioned token cost for system baselines (left) and memory mechanisms (right). Each panel uses its own linear vertical range.

5.5 COST ANALYSIS

Figure 4 compares total model tokens at the same number of native-verifier accepted nodes, preventing systems that stop early from appearing efficient merely because they complete less work. The remaining spread is large: Deep Agents averages roughly 0.31M tokens per task, whereas OMAC averages 8.01M—about 26 times more—and Peer Review averages 3.61M. These costs reflect their respective supervisor-subagent, multi-round collaboration, and create-review-revise structures. Yet Table 3 shows no corresponding dominance in SR, VHS, or ICS. Coordination overhead is therefore neither sufficient for completion nor a proxy for boundary quality; detailed accounting appears in Appendix F.

The controlled comparison makes the memory trade-off clearer. Relative to the 489k-token LOCAL-ONLY control, FULL CONTEXT uses 672k tokens (about 37% more) and SHARED FLAT RAG uses 621k (about 27% more), without consistently improving both required sharing and isolation. PlugMem is closer in cost at 521k but also varies sharply by domain. Efficient collaborative memory is therefore neither minimal context nor maximal recall: it is selective exposure that delivers the required artifact at a verified boundary without carrying an inapplicable one.

6 CONCLUSION

We introduce COMEMBENCH, an execution-grounded benchmark of collaborative memory boundaries across 800 topology-diverse workflows in four domains. Matched isolation challenges and native evaluators reveal that verified progress need not yield completion, stronger sharing does not ensure isolation, and aggregate scores conceal boundary-specific weaknesses—motivating selective, topology-conditioned memory exposure.

AI USE STATEMENT

Generative AI tools assisted with language editing, code development and debugging, figure drafting, and preliminary checks of benchmark artifacts and experimental outputs. The authors determined the research questions, benchmark design, data-selection and annotation criteria, experimental protocol, and scientific claims; they reviewed the AI-assisted material and take full responsibility for the paper and released artifacts.

REPRODUCIBILITY STATEMENT

We will release the benchmark data, clean-polluted pair specifications, schemas, evaluation code, prompts, and environment configurations. Each run records system and model versions, decoding

and budget settings, source and evaluator revisions, and asset checksums. Appendix A documents the released records, Appendix B describes construction and certification, and Appendices D–F specify the execution substrates, baseline implementations, and scoring protocol.

REFERENCES

- Saaket Agashe, Yue Fan, Anthony Reyna, and Xin Eric Wang. LLM-Coordination: Evaluating and analyzing multi-agent coordination abilities in large language models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 8053–8072, 2025. doi: 10.18653/v1/2025.findings-naacl.448. URL <https://aclanthology.org/2025.findings-naacl.448/>.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL <https://aclanthology.org/2024.acl-long.172/>.
- Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, et al. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. *arXiv preprint arXiv:2508.06600*, 2025.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- CrewAI, Inc. Crewai: Framework for orchestrating role-playing autonomous agents. <https://github.com/crewAIInc/crewAI>, 2026. Software documentation and source code.
- Ran Gong, Qiuyuan Huang, Xiaojian Ma, Yusuke Noda, Zane Durante, Zilong Zheng, Demetri Terzopoulos, Li Fei-Fei, Jianfeng Gao, and Hoi Vo. MindAgent: Emergent gaming interaction. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pp. 3154–3183, 2024. doi: 10.18653/v1/2024.findings-naacl.200. URL <https://aclanthology.org/2024.findings-naacl.200/>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=kIoBbc76Sy>.
- Jiewen Hu, Thomas Zhu, and Sean Welleck. miniCTX: Neural theorem proving with (long-) contexts. *arXiv preprint arXiv:2408.03350*, 2024. URL <https://arxiv.org/abs/2408.03350>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=ZgQ0t3zYTQ>.
- Yuntong Hu, Yuting Zhang, Matthew Trager, Yi Zhang, Shuo Yang, Wei Xia, and Stefano Soatto. EvoMAS: Evolutionary generation of multi-agent systems. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b.
- Harsh Jhamtani, Jacob Andreas, and Benjamin Van Durme. LLM agents for coordinating multi-user information gathering. In *Findings of the Association for Computational Linguistics: ACL 2025*, pp. 17800–17826, 2025. doi: 10.18653/v1/2025.findings-acl.916. URL <https://aclanthology.org/2025.findings-acl.916/>.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Zixuan Ke, Yifei Ming, Austin Xu, Ryan Chin, Xuan-Phi Nguyen, Prathyusha Jwalapuram, Jiayu Wang, Semih Yavuz, Caiming Xiong, and Shafiq Joty. MAS-Orchestra: Understanding and improving multi-agent reasoning through holistic orchestration and controlled benchmarks. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=3fGXBm4c3S>.
- LangChain AI. Deep agents: The batteries-included agent harness. <https://github.com/langchain-ai/deepagents>, 2026. Software documentation and source code.
- Dong-Ho Lee, Adyasha Maharana, Jay Pujara, Xiang Ren, and Francesco Barbieri. RealTalk: A 21-day real-world dataset for long-term conversation. *arXiv preprint arXiv:2502.13270*, 2025. URL <https://arxiv.org/abs/2502.13270>.
- Dapeng Li, Zhen Tan, Peng Qian, Yuheng Li, Kunal Chaudhary, Li Hu, and Jianwei Shen. SMOA: Improving multi-agent large language models with sparse mixture-of-agents. In *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, pp. 54–65. Springer, 2025.
- Junlong Li, Wenshuo Zhao, Jian Zhao, Weihao Zeng, Haoze Wu, Xiaochen Wang, Rui Ge, Yuxuan Cao, Yuzhen Huang, Wei Liu, Junteng Liu, Zhaochen Su, Yiyang Guo, Fan Zhou, Lueyang Zhang, Juan Michelini, Xingyao Wang, Xiang Yue, Shuyan Zhou, Graham Neubig, and Junxian He. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *The Fourteenth International Conference on Learning Representations*, 2026a. URL <https://openreview.net/forum?id=z53s5p0qhf>.
- Shijun Li, Hilaf Hasson, and Joydeep Ghosh. OMAC: A holistic optimization framework for LLM-based multi-agent collaboration. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.747. URL <https://aclanthology.org/2024.acl-long.747/>.
- Nous Research. Hermes agent. <https://github.com/NousResearch/hermes-agent>, 2026. Software documentation and source code.
- OpenAI. Codex: Lightweight coding agent that runs in your terminal. <https://github.com/openai/codex>, 2026. Software documentation and source code.
- Minjun Park, Donghyun Kim, Hyeonjong Ju, Seungwon Lim, Dongwook Choi, Taeyoon Kwon, Minju Kim, and Jinyoung Yeo. PAC-BENCH: Evaluating multi-agent collaboration under privacy constraints. In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 31030–31056, 2026. doi: 10.18653/v1/2026.findings-acl.1552. URL <https://aclanthology.org/2026.findings-acl.1552/>.
- Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. KILT: A benchmark for knowledge intensive language tasks. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 2523–2544, 2021. doi: 10.18653/v1/2021.naacl-main.200. URL <https://aclanthology.org/2021.naacl-main.200/>.

- Zhe Ren, Yibo Yang, Yimeng Chen, Zijun Zhao, Benshuo Fu, Zhihao Shu, Bingjie Zhang, Yangyang Xu, Dandan Guo, and Shuicheng Yan. GateMem: Benchmarking memory governance in multi-principal shared-memory agents. *arXiv preprint arXiv:2606.18829*, 2026. URL <https://arxiv.org/abs/2606.18829>.
- Yongliang Shen, Kaitao Song, Xu Tan, Wenqi Zhang, Kan Ren, Siyu Yuan, Weiming Lu, Dongsheng Li, and Yueting Zhuang. TaskBench: Benchmarking large language models for task automation. In *Advances in Neural Information Processing Systems*, volume 37, pp. 4540–4574, 2024. doi: 10.52202/079017-0148. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/085185ea97db31ae6dcac7497616fd3e-Abstract-Datasets_and_Benchmarks_Track.html.
- Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, 2024. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Luanbo Wan and Weizhi Ma. StoryBench: A dynamic benchmark for evaluating long-term memory with multi turns. *arXiv preprint arXiv:2506.13356*, 2025. URL <https://arxiv.org/abs/2506.13356>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=pZiyCaVuti>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, 2025.
- Zhenran Xu, Senbao Shi, Baotian Hu, Jindi Yu, Dongfang Li, Min Zhang, and Yuxiang Wu. Towards reasoning in large language models via multi-agent peer review collaboration. *arXiv preprint arXiv:2311.08152*, 2023.
- Jingbo Yang, Kwei-Herng Lai, Xiaowen Wang, Shiyu Chang, Yaari Harari, and Evgeniy Gabrilovich. GroupMemBench: Benchmarking LLM agent memory in multi-party conversations. *arXiv preprint arXiv:2605.14498*, 2026a. URL <https://arxiv.org/abs/2605.14498>.
- Ke Yang, Zixi Chen, Xuan He, Jize Jiang, Michel Galley, Chenglong Wang, Jianfeng Gao, Jiawei Han, and ChengXiang Zhai. PlugMem: A task-agnostic plugin memory module for LLM agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026b.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik R. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.
- Puzhen Zhang, Weijie Bai, Wendong Fan, and Guohao Li. Toolathlon-GYM: A large-scale environment for real-world tool-use agents. GitHub repository, 2026a. URL https://github.com/eigent-ai/toolathlon_gym.

Yuzhe Zhang, Feiran Liu, Yi Shan, Xinyi Huang, Xin Yang, Yueqi Zhu, Xuxin Cheng, Cao Liu, Ke Zeng, Terry Jingchen Zhang, and Wenyuan Jiang. SILO-BENCH: A scalable environment for evaluating distributed coordination in multi-agent LLM systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 29379–29398, 2026b. doi: 10.18653/v1/2026.acl-long.1354. URL <https://aclanthology.org/2026.acl-long.1354/>.

Wanjuan Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 19724–19731, 2024. doi: 10.1609/aaai.v38i17.29946. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29946>.

Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. MultiAgentBench: Evaluating the collaboration and competition of LLM agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8580–8622, 2025. doi: 10.18653/v1/2025.acl-long.421. URL <https://aclanthology.org/2025.acl-long.421/>.

A BENCHMARK SCHEMA AND RUNTIME PROTOCOL

The release separates reusable workflow definitions from executions. A `workflow_family` record stores the source references, global request, node-local tasks, typed dependency edges, topology witnesses, execution contract, and split. A `benchmark_instance` binds one family to either its clean control or matched polluted variant; the two variants share the family, scenario, and environment initialization. The public package contains 800 families, 1,600 instances, and 8,975 node-local contexts. Private evaluator oracles and pollution plans are distributed only with the operator package.

At execution time, a worker receives a `node_view`: its role and task, authorized local context, declared incoming artifacts, handoff contract, output type, and an opaque capability token. An artifact records its semantic type, producer, scenario and entity scope, version, verification state, payload, and provenance. The native evaluator returns a verification receipt, and only a verified receipt permits a required artifact to satisfy a downstream dependency. Systems that control routing may emit a `memory_proposal` containing an artifact, producer, and proposed recipients; the harness records the proposal and the artifacts actually exposed to each consumer.

These records describe the benchmark-visible boundary, not an architecture’s internal memory. A system may use a transcript, vector store, structured memory, or no persistent store, but private reasoning and store contents are neither inspected nor treated as ground truth. Replay instead follows the manifest-fixed case initialization, node views, native actions, receipts, issued artifacts, handoffs, consumed-artifact identifiers, and pollution events. Schema validation, content hashes, and receipt signatures make an exposure trace replayable without assuming a common internal memory representation.

B DATA SOURCES AND CONSTRUCTION DETAILS

Table 7 records the pinned sources and the evidence used to recover task dependencies. Source repositories are frozen at the listed revisions; their license and attribution files are preserved. Toolathlon-GYM is Apache-2.0, SWE-bench and BrowseComp+ are MIT licensed, and the three Lean projects retain their project-specific upstream licenses. MCP-server components remain subject to their own licenses rather than being relicensed by COMEMBENCH.

For each source record, we first build a domain-level graph and then extract a connected subgraph with one terminal objective, complete ancestor closure, and a native evaluator. No workflow joins unrelated source cases or crosses domains. Selection retains only edges with the evidence in Table 7 and favors subgraphs that instantiate the released topology motifs without adding synthetic dependencies. The request is then realized from the selected source goal, entities, resources, deliverables, constraints, and acceptance conditions; node tasks expose only the local information and declared predecessors

Table 7: Pinned sources and domain-graph recovery.

Domain	Source snapshot	Task units	Dependency evidence
Stateful tool use	Toolathlon-GYM 45bb4935	Source-authored operations over named resources and side effects	Resource flow, state transitions, evaluator predicates, and typed source-workflow dependencies
Repository code	SWE-bench Verified c104f84	Issue analysis, reproduction, localization, patching, and regression checks for one issue	Base commit, issue-test links, modified symbols, and FAIL-TO-PASS/PASS-TO-PASS contracts
Offline retrieval	BrowseComp+ 144cff8e	Query constraints, fixed-corpus evidence partitions, entity resolution, answer evidence, and synthesis	Hidden qrels, stable document identifiers, exact spans, constraint coverage, and provenance requirements
Formal mathematics	HepLean 7448822a; HTPI 8eeebaec; PNT 23650db8	Lean declarations in an elaborated target cone	Proof-term used constants, source modules and lines, and Lean 4.7 elaboration

required to perform their work. This produces a new multi-agent workflow while preserving the source task and its executable semantics.

Splits are assigned at family level, so the clean and polluted variants and all node views of a workflow remain together. Repository and Lean-project group checks additionally prevent the same repository or proof project from crossing splits. The released assignment contains 177 development and 623 test families, balanced at 200 families per domain overall.

C QUALITY ASSURANCE AND AUDIT PROTOCOL

Certification combines exhaustive machine gates with semantic review. JSON Schema validation covers families, instances, contexts, node views, artifacts, proposals, and private oracles. Graph gates check identifier closure, reachable terminal objectives, acyclicity outside declared revision edges, topology witnesses, and a dependency-evidence record on every edge. Node-local closure is checked against the task, authorized resources and capabilities, and required incoming handoffs; an undeclared path, tool, predecessor, or output semantic fails the gate. All 800 families pass the released graph and execution-contract audits.

Evaluator closure is tested by replay rather than final-text judging. The deterministic graph-and-handoff oracle accepts the known-good trace and rejects its targeted negative trace for all 800 families. Domain-native evaluators then determine node and terminal success during benchmark execution; all 200 retrieval families additionally pass document, qrel, and exact-span closure. For isolation challenges, every plan must identify a real donor, a target handoff, an invalidity reason, a local topology witness, and a deterministic observable failure predicate. The candidate gate passes for all 800 matched plans, and its single-difference manifest certifies that query, graph, evaluator, environment, and non-target inputs are unchanged.

Human review is repair-oriented rather than a majority-vote annotation task. Reviewers compare each candidate with its source record and native evaluator using four questions: whether the request is natural and complete, whether every node is executable from its local view, whether each retained edge is necessary for its consumer, and whether evaluator acceptance entails the requested outcome. A disputed item is adjudicated by rechecking the source evidence and evaluator contract; it is then repaired and rerun through all gates or excluded. We therefore do not treat an automatic language-model score as evidence of realism or report an artificial inter-annotator agreement statistic.

Finally, a public-private leakage audit rejects any public occurrence of Gold answers or patches, qrel mappings, pollution operators, target labels, failure predicates, or evaluator-only fields. Source-group split checks prevent repository and proof-project leakage, and digest comparisons detect reset drift. The final release audit reports no schema, graph, oracle-replay, source-invariant, or public leakage findings.

D NATIVE EVALUATORS AND ENVIRONMENT SETUP

D.1 MCP APPLICATION ENVIRONMENT

The application environment comprises 25 MCP servers. MCP defines the agent-facing action protocol, while the corresponding resources reside in task workspaces, PostgreSQL-backed applications, and locally managed services. Before a workflow begins, COMEMBENCH materializes its initial files and application state in a family-isolated workspace, database, and service namespace. At each node, the harness starts only the servers admitted by the node capability contract. It checks process startup and tool-schema discovery before exposing the tools, and records the arguments, native observations, and state changes of executed calls. Environment initialization, state export, evaluator invocation, and cleanup remain benchmark control operations and are not available to evaluated agents.

Table 8: MCP server inventory in the pinned application-task source environment. Runtime identifiers are normalized to readable server names.

Capability group	MCP servers	Count
Files and office documents	Filesystem, Excel, Word, PowerPoint, PDF Tools	5
Collaboration and productivity	Email, Google Calendar, Google Forms, Google Sheets, Notion, Canvas, Memory	7
Research and web content	Local arXiv, arXiv LaTeX, Scholarly Search, Fetch, Playwright, YouTube, YouTube Transcript, HowToCook	8
Business and structured data	Snowflake, WooCommerce, Yahoo Finance, 12306 Rail	4
Local computation	Terminal	1
Total		25

Server source, dependencies, build artifacts, and launch configurations are pinned before evaluation and applied uniformly to all systems. Environment certification records server endpoints, database routing, initial and final state digests, and cleanup receipts; a task with an unavailable declared server is excluded from method-level scoring.

D.2 NON-MCP NATIVE SUBSTRATES

The other execution substrates retain their native interfaces instead of being artificially wrapped as MCP services. Code outputs are submitted to the official SWE-bench evaluation endpoint, `swebench.harness.run_evaluation`, which applies the patch inside the instance-specific Docker environment and executes its `FAIL_TO_PASS` and `PASS_TO_PASS` tests. Mathematical outputs are inserted into an isolated copy of the corresponding Lean source file and checked with the pinned Lake-managed Lean 4.7 toolchain; the original source is restored after compilation. Retrieval nodes operate over immutable corpus snapshots and submit an answer with document identifiers and exact source spans. The retrieval verifier checks document existence, span provenance, declared constraint coverage, and answer correctness; it evaluates evidence selection and synthesis rather than open-web search behavior. All three substrates emit the same canonical action, artifact, receipt, and failure records used by the MCP path.

E BASELINE INSTANTIATIONS AND RUN MANIFESTS

Table 9 records the exact implementation evaluated for each system. The adapter supplies node-local tasks, authorized actions, and declared incoming artifacts, and records traces without choosing answers, routing, schedules, or memory contents.

Table 9: System baseline implementations.

Baseline	Version / revision	Evaluated instantiation
Codex	CLI 0.152.1	Official <code>codex exec -json</code> loop with isolated configuration and native event capture.
Hermes	0.21.0 (d3e2ace)	Native <code>AIAgent.chat</code> and <code>delegate_task</code> execution with sandbox tools.
Deep Agents	0.7.12 (3db758c)	Official <code>create_deep_agent</code> supervisor-subagent loop with native model and sandbox-tool bindings.
CrewAI	1.15.18 (b608a35)	Native <code>Agent/Task/Crew</code> objects with sequential process orchestration.
AutoGen	0.7.5 (027ecf0)	<code>RoundRobinGroupChat</code> with native messages and tool callbacks.
Peer Review	EvoMAS source 93fd9d6	Three creators produce candidates, followed by nine reviews, three revisions, and aggregation.
SMoA	Source 1f3444d	Official sparse-mixture process with its role layers and aggregation policy preserved.
OMAC	Source 6bccfec	Frozen five-dimensional, three-round LLMLP package; optimized functions and collaboration structure are retained.
ADAS	Source 2702bee	The selected agent program is frozen before benchmark evaluation and executed through the official agent interface.
EvoMAS	Source 93fd9d6; smolagents 1.26.0	Frozen evolved role, prompt, and organization configuration executed by <code>MasRuntime</code> .

Controlled memory mechanisms. All six mechanisms run in the same supervisor-worker executor. LOCAL-ONLY exposes only worker-local state; FULL CONTEXT exposes every prior visible artifact without retrieval; and SHARED FLAT RAG retrieves the top four artifacts from one workflow-local store using `all-MiniLM-L6-v2`. Mem0 0.1.104 uses its local `add/search` API with a workflow-specific Qdrant store. A-Mem (source 0c8039f) writes verified artifacts as linked notes and retrieves related notes for each node. PlugMem (source 3b2ce75) uses the official graph write, recall, and reasoning path. Each external store is empty at workflow start; only the memory policy changes across rows.

Every run has an immutable manifest recording source and adapter revisions, model and decoding settings, budgets, configuration hashes, memory policy, data split, environment and evaluator hashes, seed, and trace schema. This separates a baseline’s organization from the common execution and scoring substrate.

F METRIC DEFINITIONS AND SCORING DETAILS

For clean controls $\mathcal{C}$, SR is $|\mathcal{C}|^{-1} \sum_{q \in \mathcal{C}} \mathbb{I}[\text{terminal_success}(q)]$. Let $\mathcal{U}_q$ be the canonical required nodes of q and $\mathcal{V}_q \subseteq \mathcal{U}_q$ those with a verified completion; VNCR is $|\mathcal{C}|^{-1} \sum_{q \in \mathcal{C}} |\mathcal{V}_q|/|\mathcal{U}_q|$. Repeated executions and revision retries count once, while method-private nodes never enter either set.

For VHS, let $\mathcal{H}$ be non-root consumers whose mandatory direct predecessors have verified successfully, and let $\mathcal{H}^+$ be the successful subset. A consumer enters $\mathcal{H}^+$ only if every required predecessor artifact is exposed at the declared boundary, current, receipt-valid, explicitly consumed, and followed by native-verifier success. Thus VHS is $|\mathcal{H}^+|/|\mathcal{H}|$; unsupported slices are marked “-”.

Each isolation pair z declares one target consumer t_z . Let $\mathcal{P}$ contain pairs for which t_z succeeds in the clean variant, and $\mathcal{P}^+$ those for which it also succeeds in the matched polluted variant. Thus ICS is $|\mathcal{P}^+|/|\mathcal{P}|$; eligibility is not terminal success of the full clean workflow. Clean and polluted variants share the query, graph, initial state, tools, and non-target inputs; only one evaluator-hidden inapplicable artifact at t_z differs.

Token accounting includes execution, coordination, memory-retrieval context, recovery or repair, and meta-level search or evolution. We report input/output tokens, mean, median, 90th percentile, successful-run mean, and total valid-run tokens divided by successful workflows. Benchmark-visible artifact exposure is reported separately. Conditional measures are computed only over eligible cases. Benchmark and infrastructure failures are excluded from method-failure denominators.

G ADDITIONAL EVALUATION DETAILS

Table 10 reports the aggregate token statistics behind Figure 4. Values include model calls used for execution, coordination, recovery, and memory operations and are computed over runs with complete usage telemetry.

Table 10: Token accounting in thousands of tokens.

System	Mean	Median	Per verified node	Memory mechanism	Mean	Median	Per verified node
Codex	2350	1875	1098	Local-only	489	367	80
Hermes	518	382	194	Full Context	672	380	104
Deep Agents	310	227	89	Shared Flat RAG	621	389	94
CrewAI	461	263	112	Mem0	576	422	91
AutoGen	388	231	97	A-Mem	617	357	102
Peer Review	3608	2265	1022	PlugMem	521	392	90
SMoA	1419	858	333				
OMAC	8012	5536	1981				
ADAS	565	436	290				
EvoMAS	1306	977	343				

Table 11 states the failure attribution applied before metric aggregation. Only method failures enter method-performance denominators; benchmark and infrastructure failures remain separately auditable.

Table 11: Run-outcome attribution.

Outcome	Criterion	Scoring treatment
Success	Native evaluator accepts the required terminal state or target node	Positive outcome
Method failure	A valid run produces a rejected action/state, exhausts its protocol, or terminates without an admissible result	Included as failure
Benchmark failure	Released asset, schema, reset, oracle, or evaluator contract is inconsistent	Quarantined
Infrastructure failure	Required service, container, endpoint, or evaluator process is unavailable independently of system behavior	Excluded and rerun